SmartGate – IoT Smart Intercom System

Moderni IoT sistem za pametnu kontrolu pristupa zgradama zasnovan na Raspberry Pi uređaju, MQTT protokolu, Spring Boot backendu i Angular web aplikaciji.

Sadržaj

- Opis projekta
- Vrednosna ponuda i benefiti za korisnike
- Business Model
- Arhitektura sistema
- Tehnologije
- Komponente sistema
  - IoT sloj – Raspberry Pi (Python)
  - Backend – Spring Boot
  - Frontend – Angular
- Tok podataka
- Konfiguracija
- Pokretanje sistema
- API referenca
- WebSocket / STOMP
- MQTT topici
- Status projekta
- Preporuke za monetizaciju i naredne korake
- Buduća proširenja

Opis projekta

SmartGate je IoT sistem za pametnu kontrolu pristupa koji zamenjuje klasične interfone. Kada posetilac pritisne dugme ispred zgrade, Raspberry Pi uređaj:

1. Automatski snima fotografiju posetioca putem kamere (OpenCV)
2. Uploaduje sliku na Spring Boot backend
3. Prosleđuje event stanar putem MQTT → WebSocket bridge-a
4. Stanar prima real-time notifikaciju u Angular web aplikaciji sa slikom posetioca
5. Jednim klikom stanar otvara kapiju – komanda se šalje nazad kroz MQTT do releja

Sistem podržava više stanova (multi-apartment), radi u lokalnoj mreži i projektovan je da bude skalabilan.

Vrednosna ponuda i benefiti za korisnike

SmartGate donosi jasnu vrednosnu ponudu: povećana bezbednost i komfor u upravljanju pristupom. Ključni benefiti su:

Poboljšana bezbednost: Stanarima se šalje fotografija posetioca pre otvaranja, što omogućava vizuelnu verifikaciju i sprečava neovlašćen ulazak.

Ovo daje stanarima mir jer mogu da kontrolišu ko ulazi čak i kad su odsutni.

Lakoću upotrebe i bolji UX: Daljinsko upravljanje kapijom sa bilo kog mobilnog uređaja štedi vreme. Stanari ne moraju da prilaze interfonskoj tabli – mogu otvoriti ulaz iz udobnosti stana ili sa puta (kao što je opisano kod sličnih IoT interfon sistema). Dvosmerni audio i video prenos obezbeđuje prirodnu komunikaciju sa posetiocima.

Skalabilnost i modularnost: Za razliku od tradicionalnih rešenja, SmartGate sistem lako se širi na više stanova i ulaza korišćenjem mrežne infrastrukture (MQTT). Nema potrebe za zasebnom instalacijom za svaki korisnik – novi moduli se samo dodaju u IoT mrežu. Sistem podržava višestruke pozivne jedinice i stanice i može se nadograditi dodatnim funkcionalnostima bez velikih infrastrukturnih izmena.

Niži troškovi: SmartGate koristi dostupne komponente (Raspberry Pi, standardne kamere, open-source softver), što značajno snižava cenu po jedinici. Tako je rešenje cenovno povoljnije od komercijalnih interfon sistema (koji uključuju skupe centralne jedinice i licencirane softvere).

Uz to, otvorena arhitektura smanjuje troškove održavanja jer se oslanja na široko podržanu zajednicu hardvera i softvera.

Integracije i fleksibilnost: IoT platforma omogućava povezivanje sa drugim pametnim sistemima. Na primer, sistem lako može komunicirati sa sigurnosnim kamerama, alarmima ili pametnim bravama zgrade preko standardnih protokola (MQTT, REST). Prema autoru ButterflyMX, „IoT interkomi mogu se integrisati sa softverom za upravljanje imovinom i drugim pametnim rešenjima“.

Tako SmartGate može postati deo šireg pametnog ekosistema (npr. upravljanje liftom ili senzorima).

Ciljni segmenti i primeri upotrebe: SmartGate je pogodan za razne korisnike:

Stambene zgrade i upravnice: Upravnici višespratnica mogu implementirati SmartGate na ulazima i garažama. Stanari tada dobijaju sliku posetioca i otvaraju kapiju putem mobilne aplikacije ili web interfejsa. Ovo je posebno korisno u velikim zgradama sa više ulaza, gde stanari ne moraju da trče do interfona na svakom spratu.

Mali poslovni objekti i kancelarije: Office objekti često imaju recepciju. SmartGate omogućava vlasnicima i zaposlenima da pregledaju ko dolazi (kuriri, klijenti) i daju im pristup bez fizičkog prisustva. Time se olakšava logistika i povećava sigurnost imovine.

Privatne kuće sa kapijom: Vlasnici kuća mogu instalirati SmartGate na kapiji ili ulazu. Čak i kada su van kuće, mogu pustiti dostavljače ili goste koristeći aplikaciju – a svi pokušaji ulaska se beleže.

Co-working i poslovni prostori: U zajedničkim radnim prostorima, SmartGate može kontrolisati pristup članovima, gostima i kuririma. Administrator brine da samo ovlašćeni korisnici ulaze, a čitači pametnih kartica ili QR kodova mogu biti dodatne opcije (RFID integracija).

Ovi segmenti cene sigurnost, pouzdanost i fleksibilnost. SmartGate ih ispunjava tako što kombinuje IoT funkcionalnost sa jednostavnim korisničkim iskustvom i niskim operativnim troškovima.

Business Model

Ključni elementi poslovnog modela SmartGate sistema su:

Ključni partneri: Proizvođači hardvera (Raspberry Pi, kamere, releji), dobavljači elektronike, instalaterske firme za implementaciju, IoT/softver zajednica (za podršku open-source alata).

Ključne aktivnosti: Razvoj softvera (IoT protokola, backenda, frontend aplikacije), proizvodnja/pakovanje uređaja, instalacija kod korisnika, tehnička podrška i ažuriranje sistema.

Ključni resursi: Softver (firmware, cloud/backend, aplikacije), stručni tim (IoT inženjeri, programeri), hardverski prototipovi i alati za montažu, znanje iz bezbednosnih sistema.

Vrednosna ponuda: Sigurnost i kontrola pristupa sa slike, daljinski nadzor i otvaranje kapije preko mobilnog uređaja, jednostavna instalacija i skalabilnost. SmartGate kombinuje IoT inovacije sa praktičnim upravljanjem zgradom, uz niži TCO zbog jeftine komponente (Raspberry Pi) i open-source softvera.

Odnos sa kupcima: Prvenstveno B2B/B2G model – saradnja sa upravnicima zgrada, firmama za obezbeđenje i stambenim zadrugama. Održavanje uključuje tehničku podršku i ažuriranja. Moguće je i B2C prodaja za kućna rešenja, uz online kanale i direktnu prodaju.

Kanali distribucije: Direktna prodaja i instalacija preko partnera (npr. firme koje postavljaju interfonske sisteme), online prodavnica/opšte IT/IoT kanali, te sajmovi i konferencije o pametnim zgradama.

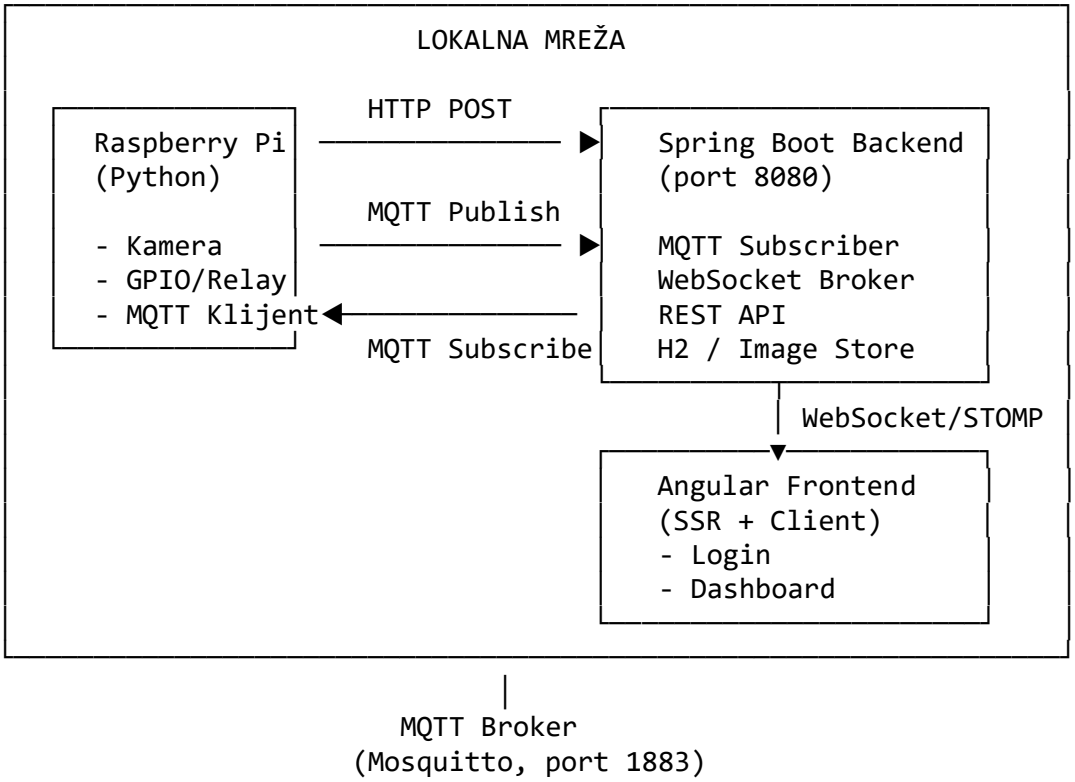
Segmenti kupaca: Stanodavci i upravnici zgrada, male i srednje firme sa kontrolisanim ulazom, privatni vlasnici kuća sa kapijama, i co-working prostori. Svaki segment je orijentisan na veću bezbednost i praktičnost upravljanja pristupom.

Struktura troškova: Troškovi hardvera (Raspberry Pi, kamere, releji), razvoj softvera (plaće programera i IoT stručnjaka), operativni troškovi (server hosting za cloud deo ako se koristi), marketing i podrška korisnicima. Korišćenje otvorenih tehnologija smanjuje licence i dugoročne izdatke.

Tokovi prihoda: Prodaja uređaja (ceo komplet ili pojedinačno), pretplata (SaaS) za dodatne cloud usluge (npr. čuvanje istorije događaja ili video zapisa), pretplata na premium funkcije (npr. AI

prepoznavanje lica), kao i licenciranje rešenja drugim integratorima. Može se razmotriti i B2B model „Enterprise license“ za veće objekte.

Arhitektura sistema



Tehnologije

IoT sloj

Komponenta	Tehnologija
Platforma	Raspberry Pi (Linux)
Jezik	Python 3
Kamera	OpenCV (cv2)
MQTT klijent	paho-mqtt
GPIO upravljanje	gpiozero
HTTP upload	requests

Backend

Komponenta	Tehnologija
Framework	Spring Boot **4.0.5**
Java verzija	**Java 26**

Komponenta	Tehnologija
Build alat	Maven
MQTT integracija	Spring Integration MQTT + Eclipse Paho 1.2.5
WebSocket	Spring WebSocket + STOMP
Persistencija	Spring Data JPA
Baza podataka	H2 (in-memory, dev)
Serializacija	Jackson Databind
Validacija	Spring Boot Validation

Frontend

Komponenta	Tehnologija
Framework	Angular (standalone komponente)
Arhitektura	SSR (Server-Side Rendering) + klijentska aplikacija
WebSocket klijent	@stomp/stompjs
HTTP komunikacija	Angular HttpClient
Stilizacija	CSS (per-komponenta)
Routing	Angular Router

Infrastruktura

Komponenta	Tehnologija
Message broker	Mosquitto MQTT (port 1883)
Slike na disku	Lokalni uploads/ direktorijum
Serviranje statičkih fajlova	Spring MVC Resource Handler

Komponente sistema

IoT sloj – Raspberry Pi (Python)

Raspberry Pi je centralna IoT jedinica sistema. Modularna je struktura sledećih servisa:

Struktura projekta (IoT)

```
iot/
├── main.py              # Entry point
├── config/
│   └── settings.py      # Konfiguracija (broker IP, pinovi, topic-i)
├── camera/
│   └── camera_service.py   # Snimanje slike (OpenCV)
├── gpio/
│   ├── button_handler.py   # Dugmad (GPIO ili konzolni test mode)
│   └── relay_controller.py # Upravljanje relejom (gpiozero)
├── mqtt/
│   ├── mqtt_client.py      # MQTT klijent (paho-mqtt, MQTTv3.1.1)
│   └── topics.py           # Centralizovane MQTT topic konstante
```

```
├── services/
│   ├── intercom_service.py  # Centralna business logika
│   └── upload_service.py    # HTTP upload slike na backend
└── utils/
    └── logger.py            # Timestampovano logovanje
```

Tok izvršavanja

1. ButtonHandler detektuje pritisak dugmeta (GPIO pin ili konzolni TEST_MODE)
2. Poziva IntercomService.handle_ring(stan_id) – pokreće asinhronizovani thread
3. CameraService snima fotografiju putem OpenCV i čuva je lokalno (images/)
4. UploadService šalje sliku na backend endpoint POST /api/upload i dobija URL
5. Lokalna slika se briše; MQTT poruka sa JSON payload-om se publikuje na interfon/pozivi
6. MQTTClient se pretplaćuje na interfon/kapija/komande; po prijemu {"type":"OPEN"} aktivira relej 1 sekundu

Konfiguracija (settings.py)

```
BROKER_IP = "localhost"          # IP MQTT brokera
PORT = 1883
TOPIC_POZIVI = "interfon/pozivi"
TOPIC_KOMANDE = "interfon/kapija/komande"
CLIENT_ID = "raspberry_pi_interfon"
GPIO_BUTTONS = [2, 3, 4, 17]    # GPIO pinovi za dugmad po stanovima
RELAY_PIN = 18                  # Pin za relej (kapija)
CAMERA_INDEX = 0                # Index kamere (0 = prva)
TEST_MODE = True                # True = konzolni unos umesto GPIO
```

Test mode (TEST_MODE = True): sistem se može pokrenuti i testirati na bilo kom računaru bez fizičkih GPIO pinova ili kamere (simulator loguje akcije).

Backend – Spring Boot

Struktura projekta

```
src/main/java/com/example/demo/
├── config/
│   ├── MqttConfig.java        # MQTT inbound/outbound integration flow
│   ├── WebConfig.java         # Serviranje statičkih slika (/images/**)
│   └── WebSocketConfig.java   # STOMP WebSocket konfiguracija (/ws-intercom)
├── controller/
│   ├── GateController.java    # POST /api/open – šalje OPEN komandu
│   └── UploadController.java  # POST /api/upload – prima sliku
├── dto/
│   ├── IntercomEventDto.java  # DTO za MQTT event (stan, image_url, timestamp)
│   └── UploadResponse.java    # DTO odgovor sa URL-om slike
├── model/
│   └── ImageRecord.java       # JPA entitet za čuvanje metapodataka slike
├── repository/
│   └── ImageRepository.java   # Spring Data JPA repozitorijum
└── service/
```

```
├── ImageService.java      # Čuvanje slike na disk + DB zapis
└── MqttService.java       # MQTT poruke → WebSocket forward
```

Ključni tokovi

Upload slike: - POST /api/upload prima multipart/form-data - Fajl se čuva u uploads/ sa UUID prefiksom - Metapodaci (ime, putanja, vreme) se upisuju u H2 bazu - Vraća JSON { "url": "http://<ip>:8080/images/<filename>" }

MQTT → WebSocket bridge: - MqttConfig definiše IntegrationFlow koji sluša topic interfon/pozivi - Paho klijent prima poruku → MqttService.receive() je deserijalizuje u IntercomEventDto - SimpMessagingTemplate prosleđuje DTO na WebSocket destination /topic/intercom

Otvaranje kapije: - Frontend poziva POST /api/open - GateController → MqttService.sendOpenCommand() publikuje {"type":"OPEN"} na interfon/kapija/komande - Raspberry Pi prima komandu i aktivira relej

Konfiguracija (application.properties)

```
server.port=8080
server.address=0.0.0.0

# H2 in-memory baza
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Upload
file.upload-dir=uploads
spring.servlet.multipart.max-file-size=5MB

# MQTT
mqtt.broker=tcp://192.168.100.112:1883
mqtt.client-id=spring-backend
mqtt.topic.events=interfon/pozivi
mqtt.topic.commands=interfon/kapija/komande
```

Frontend – Angular

Struktura projekta

```
src/
├── app/
│   ├── models/
│   │   └── intercom-event.ts      # TypeScript interfejs za event
│   ├── pages/
│   │   ├── login/
│   │   │   ├── login.html         # UI forme za prijavu
│   │   │   ├── login.css
```

```
│   │   │   └── login.ts          # Standalone komponenta, navigacija na dashboard
│   │   └── dashboard/
│   │       ├── dashboard.html     # Prikaz slike, info, dugmad
│   │       ├── dashboard.css
│   │       └── dashboard.ts       # WebSocket pretplata, open/reject logika
│   ├── services/
│   │   └── intercom.ts            # IntercomService: WebSocket + HTTP
│   ├── app.routes.ts              # Routing (/ → login, /dashboard)
│   ├── app.config.ts
│   └── app.ts
└── main.ts                        # bootstrapApplication entry point
```

Ključne funkcionalnosti

Login: Jednostavna stranica sa unosom korisničkog imena i lozinke. Po prijavi postavlja localStorage['loggedIn'] i rutira na /dashboard.

Dashboard: - Inicijalizacija STOMP klijenta (@stomp/stompjs) na ws://localhost:8080/ws-intercom - Pretplata na /topic/intercom – svaki novi event ažurira prikaz slike, broja stana i vremena - Dugme **"Otvori"** → POST /api/open → kapija se otvara - Dugme **"Ne otvaraj"** → lokalna poruka, bez akcije na backendu - ChangeDetectorRef.detectChanges() osigurava UI refresh van Angular zone-e

IntercomEvent model:

```
export interface IntercomEvent {
  id?: number;
  stan?: number;
  image_url: string;
  timestamp: string;  // format "HH:mm:ss"
}
```

Tok podataka

```
[Dugme na interfonu]
        │
        ▼
[Raspberry Pi – ButtonHandler]
        │
        ▼
[CameraService – OpenCV snima sliku]
        │
        ▼
[UploadService – POST /api/upload → backend]
        │          │
        │          ▼
        │   [ImageService – čuva na disk + H2 DB]
        │          │
        │          ▼
        │   [Vraća image URL]
```

```
      │
      ▼
[MQTT Publish → interfon/pozivi]
{
  "stan": 3,
  "image_url": "http://192.168.x.x:8080/images/uuid_filename.jpg",
  "timestamp": "14:32:07"
}
      │
      ▼
[Spring Boot – MqttConfig IntegrationFlow prima poruku]
      │
      ▼
[MqttService.receive() → deserijalizuje u IntercomEventDto]
      │
      ▼
[SimpMessagingTemplate → /topic/intercom (WebSocket STOMP)]
      │
      ▼
[Angular Dashboard – prima event, ažurira UI]
      │
  ┌───┴──────────┐
  │              │
[Otvori]   [Ne otvaraj]
  │
  ▼
[POST /api/open]
  │
  ▼
[MqttService.sendOpenCommand()]
  │
  ▼
[MQTT Publish → interfon/kapija/komande]
{"type":"OPEN"}
  │
  ▼
[Raspberry Pi – RelayController.open_gate() → 1s impuls]
```

Pokretanje sistema

Preduslovi

- Mosquitto MQTT broker pokrenut lokalno ili na mreži (port 1883)
- Java 26, Maven
- Python 3.9+, pip
- Node.js + Angular CLI (za frontend)

1. MQTT Broker

```
# Instalacija (Linux)
sudo apt install mosquitto mosquitto-clients
sudo systemctl start mosquitto
```

2. Spring Boot Backend

```
cd backend/
mvn spring-boot:run
# Backend dostupan na http://0.0.0.0:8080
# H2 konzola: http://localhost:8080/h2-console
```

3. IoT – Raspberry Pi

```
cd iot/
pip install paho-mqtt opencv-python requests gpiozero
python main.py
# U TEST_MODE: unesi broj stana u konzoli
```

4. Angular Frontend

```
cd intercom-frontend/
npm install
ng serve
# Dostupno na http://localhost:4200
```

API referenca

POST /api/upload

Prima sliku sa Raspberry Pi i čuva je na server.

Request: multipart/form-data, polje file (JPEG/PNG, max 5MB)

Response:

```
{
  "url": "http://192.168.100.109:8080/images/uuid_filename.jpg"
}
```

POST /api/open

Šalje komandu za otvaranje kapije putem MQTT.

Response: 200 OK – "Kapija komanda poslata"

GET /images/{filename}

Servira sačuvanu sliku direktno iz uploads/ direktorijuma.

WebSocket / STOMP

Parametar	Vrednost
Endpoint	ws://localhost:8080/ws-intercom
Protokol	STOMP over WebSocket
Subscribe destination	/topic/intercom
Poruka	JSON IntercomEventDto

Primer poruke:

```
{
  "stan": 3,
  "image_url": "http://192.168.100.109:8080/images/abc_stan_3_143207.jpg",
  "timestamp": "14:32:07"
}
```

MQTT topici

Topic	Direkcija	Sadržaj
interfon/pozivi	RPi → Backend	JSON event (stan, image_url, timestamp)
interfon/kapija/komande	Backend → RPi	JSON komanda {"type":"OPEN"}

Status projekta

Komponenta	Status
IoT – kamera + upload	✅Implementirano
IoT – MQTT publish	✅Implementirano
IoT – relej (GPIO)	✅Implementirano (sa simulacijom za dev)
Backend – upload API	✅Implementirano
Backend – MQTT bridge	✅Implementirano
Backend – WebSocket STOMP	✅Implementirano
Frontend – login	✅Implementirano
Frontend – dashboard + real-time	✅Implementirano
Autentikacija (JWT/OAuth)	☐ Planirana
Produkciona baza (PostgreSQL)	☐ Planirana

Preporuke za monetizaciju i naredne korake

SaaS pretplata: Ponuditi korisnicima mesečni plan za oblak i napredne usluge (backup podataka, centralni dashboard za više lokacija, analitiku). Ovo obezbeđuje konstantan prihod i omogućava kontinuirano unapređenje softvera.

Premium funkcionalnosti: Razviti dodatne opcije koje se plaćaju: napredni video-algoritmi (prepoznavanje lica ili registarskih tablica), pametno upravljanje višestrukim ulazima, integracije sa kućnim asistentskim sistemima (npr. Google Home, Alexa). Nalazimo paralelu sa ButterflyMX koji dodatno naplaćuje bogatije funkcije.

B2B prodaja i partnerstva: Fokusirati se na prodaju velikim kupcima – upravnicima stambenih zgrada, firmama za obezbeđenje i kompanijama za kontrolu pristupa. Moguće je licencirati rešenje proizvođačima hardvera ili pružati „white-label“ uslugu. Takođe, razmotriti pilot-projekat sa lokalnom samoupravom za javne stanove ili škole.

Integracije: Umrežiti SmartGate sa drugim sistemima: pametni gradovi, IoT platforme ili ERP/CRM sisteme zgrada. Na primer, automatsko prijavljivanje servisnih poseta u zajedničkom kalendaru.

Pilot projekti i validacija: Krenuti sa pilot-implementacijom u jednoj ili dve stambene zgrade kako bi se demonstrirale prednosti i prikupili povratni podaci. Na osnovu pilot-faze, izraditi verziju MVP (sa nužnim funkcionalnostima), zatim testirati na manjoj grupi korisnika. Poželjno je „prilagoditi“ rešenje na osnovu povratnih informacija pre šire komercijalizacije.

Marketing i sertifikacije: Dobiti relevantne sertifikate (npr. CE) i uveriti korisnike u bezbednost rešenja. Komunicirati prednosti preko studija slučaja (case studies) i demonstracionih video-snimaka.

Buduća proširenja

JWT autentikacija
Uvođenje JWT autentikacije omogućava zaštitu API endpointa i WebSocket konekcija od neovlašćenog pristupa. Na ovaj način se obezbeđuje da samo registrovani korisnici mogu da pristupe sistemu i upravljaju kapijom. Ovo je ključan korak ka produkcionoj verziji sistema gde bezbednost ima prioritet.

PostgreSQL baza podataka
Zamena H2 in-memory baze sa PostgreSQL bazom omogućava trajno čuvanje podataka i veću pouzdanost sistema. PostgreSQL je skalabilno i robusno rešenje koje je standard u produkcionim okruženjima. Time sistem postaje spreman za realnu upotrebu i veći broj korisnika.

Istorija poseta
Implementacija istorije poseta omogućava pregled svih dolazaka po stanovima, uključujući slike

i vreme posete. Ova funkcionalnost povećava bezbednost jer korisnici mogu naknadno proveriti ko je dolazio. Takođe, može biti korisna za administraciju i evidenciju pristupa.

Cloud storage (Amazon S3 ili slično)
Prebacivanje skladištenja slika na cloud omogućava veću dostupnost i sigurnost podataka. Umesto lokalnog čuvanja, slike se mogu skladištiti na distribuiranim serverima sa backup opcijama. Ovo je posebno važno za skaliranje sistema i pristup podacima sa više lokacija.

Face recognition (AI prepoznavanje lica)
Integracija AI sistema za prepoznavanje lica omogućava automatsko prepoznavanje poznatih posetilaca. Sistem može sam odlučiti da li da dozvoli ulaz bez intervencije korisnika. Ova funkcionalnost značajno podiže nivo automatizacije i bezbednosti.

Push notifikacije (Firebase Cloud Messaging)
Uvođenjem push notifikacija korisnici dobijaju obaveštenja direktno na mobilne uređaje u realnom vremenu. Ovo omogućava bržu reakciju čak i kada korisnik nije aktivno u aplikaciji. Time se poboljšava korisničko iskustvo i dostupnost sistema.

Multi-user podrška
Omogućavanje više korisnika po stanu uvodi fleksibilnost u upravljanju pristupom. Različiti korisnici mogu imati različita prava, na primer članovi porodice ili zaposleni u firmi. Ova funkcionalnost je neophodna za realne scenarije korišćenja.

Video stream (RTSP/HLS)
Zamena statičkih slika live video stream-om omogućava korisnicima da u realnom vremenu vide posetioca. Ovo daje kompletniju sliku situacije i dodatno povećava sigurnost. Implementacija RTSP ili HLS protokola omogućava efikasan i stabilan prenos videa.